

Rwanda MINECOFIN Quarterly Macro-Fiscal Model infrastructure

Operator's manual

December 31, 2023

Section contents

Overview

Basic features

- The operating infrastructure of the Quarterly Macro-Fiscal Model (QMFM) is designed to facilitate forecasting and scenario analysis, as well as model calibration and evaluation
- A key feature is the ability to effortlessly create comparison reports
- The comparisons can be performed along various dimensions: parametrization, scenario assumptions, forecast rounds, etc.
- The results include, among others, equation decompositions, impulse responses, variance decompositions, forecast change decompositions, and historical forecast performance analysis
- When used properly, the infrastructure ensures reproducibility of earlier results, while at the same time allowing for continuous model and code development

Software environment

- The infrastructure is implemented in Matlab, using the IRIS macroeconomic modeling toolbox
- Matlab version 2020b and IRIS version 20230622 are required
- The input data is stored in Excel files
- Results are saved in Matlab MAT-files, and also in CSV files (the latter can be opened in Excel as well)
- Reports are generated as PDF files
- It is advisable to use a PDF viewer that does not lock the file (so that the reports can be regenerated while open)
 - A example is Sumatra PDF Reader

Folder structure: main QMFM folder

- All files must be in the *main QMFM folder*; the name of this folder is at the users discretion
- Within the main QMFM folder, there must a subfolder with the name `data`; all data files must be placed here
- Besides the `data` folder, the main QMFM directory must contain *round branch folders*
- While the names of the round branch folders are discretionary, it is recommended—in order to facilitate collaboration—, to have at least two branches called `main` and `develop`

Folder structure: the round folders

- The *round folder* contains all necessary files to run a round
- Each round folder must be placed in a round branch folder
- Within the main folder we have the following subfolders
 - `+codes` contains the Matlab M-files of the round
 - `model` contains the model file and the parameter functions
 - `tunes` contains the historical and scenario specific judgements (functions)
- Besides, there are two files here
 - `mainSettings.m` contains *all* user-defined settings
 - `mainDriver.m` executes the codes in appropriate order

The results folder

- All results are saved in (subfolders of) the `results` folder
- The `results` folder will be created automatically if it does not exist
- The `results` folder can be placed either inside the round folder, or outside, as a folder directly below the main QMFM folder
- In the second case, the results are saved in subfolders named by the round folder (including the round branch)
- The choice between the two options is governed by `opts.resultsLocation`, which can be either `'inside'` or `'outside'`
- If results are saved inside the round folder, it is advisable to delete the `results` folder before the round is shared via email, as this will drastically reduce the mail size

Workflow

- When working with a round, Matlab's current working directory must be the main folder of the round
 - Type `cd` in the Matlab command window to see the current working directory
 - Type `cd(<folder name>)` to set the current working directory to `<folder name>`
- Most of the time, the operator does only two things:
 - Adjust `mainSettings.m` according to the task at hand
 - Run `mainDriver.m` to execute the selected commands with the adjusted settings

Caveats

- Do not place the main QMFM folder into a folder which is automatically synchronized, e.g. by Google Drive, Dropbox, etc.
- Do not change the MATLAB path, except when adding the path to the IRIS toolbox. All codes are placed in MATLAB *package folders* (with names starting with a + sign), so that they can be run by typing e.g. `codes.filterHistory`
- Keep the `main` folder branch clean: it should only contain the main forecast rounds. Once a forecast round has been finalized, the codes should never be changed
- If you want to experiment with settings, preserve a certain state of a round, etc., copy rounds to the `develop` round branch
- You can even create your own personal round branch

Section contents

The mainSettings file

Structure of mainSettings

- All settings/options are given in `mainSettings`
- All options are given as fields of the `opts` structure, or its action specific substructures
- There are for main type of options

General options Names of the input files (model, parameters, data), current and comparison round specification

Run options Which action and report components to include when running `mainDriver`

Typical forecast options Filter and forecast range, chart and tables ranges, scenario names

Report options List of variables, various ranges, number of contributions, and many others

Settings: basic structure of mainSettings.m

- The file is divided into sections by lines starting with %% signs
- The settings are stored in a Matlab structure `opts`
- The settings pertaining to a particular action are collected in fields within `opts`, e.g. `opts.reportHistory`
- Related settings may further be grouped into subfields, e.g. `opts.reportModel.IRF`
- Some time-consuming or unnecessary calculations can be bypassed, e.g. setting `opts.reportHistory.eqDecomps.run = false` will not create the historical decomposition charts

Settings: general options I

- `opts.roundId`: round identifier, it will be shown in the report titles and headings
- `opts.modelfile`: name of the file containing the model equations
 - Must not contain the extension, which will automatically be assumed to be `.model`
 - The model file must be in the `model` subfolder of the main folder
 - It is not possible to compare results from different models; however, in many cases it is possible to formulate "model differences" as different parameterizations of the same model

Settings: general options II

- `opts.mainDataFile` Identifier of main data file; the file must be called `Rwametric_<opts.mainDataFile>`, and it must be located in `QMFM/data`
- `opts.addDataFile` Same as above, but for the additional data file `addData_<opts.addDataFile>` (if any)
- `opts.compRound` Name of the round to which results of the current round are compared; it must include the round branch, e.g. `main/2023-07-July`
- `opts.resultsLocation` The location of the results, see slide 7

Settings: general options III

- `opts.parameterNames`: a list of names of parameter files
 - The names must not contain the `.m` extension
 - Each parameter file is a Matlab function, returning the parameter values in a parameter database `p`
 - The parameter files must be in the `model` folder
 - Related parameter files can be collected into subfolders of the `model` folder, in which case the name must contain the subfolder name, e.g. `["subFolder/setparam1", "subFolder/setparam2"]`
- `opts.parameterLegends`: a list of short, informative descriptions of the parameter settings
 - It will appear in legends, chart title, etc.
 - Should not be very long, otherwise it may cover the chart content

Settings: run settings

- `opts.<action name>.run` determines whether the `mainDriver` will execute that action
- The value can be either `true` or `false`
- For the `reportModel` and `reportHistory` actions, certain subtasks can also be turned-off
- Regardless of the settings here, we can always run an action by executing `codes.<action name>` in the command window

Settings typical forecast round settings I

- `opts.filter.range` Range over which the Kalman filter will be run
- `opts.forecast.range` Range over which the forecast will be calculated
- `opts.forecast.scenarioNames` List of scenarios for which the forecast will be calculated
 - For each `<scenario>` in this list, the file `<scenario>.m` must exist in the `tunes` subfolder of the main folder
- `opts.forecast.scenarioLegends` A list of short, informative descriptions of the scenario settings
 - It will appear as legends in the forecast report
 - Should not be very long, otherwise it may cover the chart content

Settings typical forecast round settings II

- `opts.forecast.tableRange` Range of the forecast plots; besides the forecast range, it may (should) include the recent history as well
- `opts.forecast.plotRange` Range of the forecast plots; besides the forecast range, it may (should) include the recent history as well
- `opts.forecast.highlightRange` Range of the forecast plots which will be marked by a different background color; it should either be the forecast range, or the recent history
- `opts.forecast.tableRange_ann` Range of the annual forecast tables; besides the forecast range, it may (should) include the recent history as well

Settings: `opts.steady`

- `opts.steady.variables` A list of model variables whose steady state values will be included in the report

Settings: `opts.irf`

- `opts.irf.horizon` Horizon of the impulse response functions. The impulse happens in period 0
- `opts.irf.shocks` List of shocks to include in the report. For each shock there will be a new page in the report showing the responses to the shock.
- `opts.irf.shockSize` Size of the impulse; set it to `@auto` to use one standard deviation shocks
- `opts.irf.variables` List of variables whose responses will be plotted
- `opts.irf.subplot` A numeric vector `[r, c]` of row and column size of the grid of responses. The product `rc` must be at least as large as the number of variables

Settings: `opts.varDecomp`

- `opts.varDecomp.horizon` Horizon of the variance decomposition charts
- `opts.varDecomp.variables` List of variables whose decomposition will be plotted
- `opts.varDecomp.histRange` An IRIS date range over which the historical standard deviations will be calculated
- `opts.varDecomp.contribs` An integer `n` indicating how many shocks to name individually
 - The contribution of the `n` most important shocks will be plotted separately, other shock will be grouped together
 - The importance is determined based of the relative contribution on the first period
 - When comparing multiple parameterizations, all shocks which are important in any one parametrization will be plotted separately

Settings: `opts.stdComp`

- `opts.stdComp.variables` List of variables to be included in the standard deviations comparison table
- `opts.stdComp.shocks` List of shocks to be included in the standard deviations comparison table
- `opts.stdComp.histRange` An IRIS date range over which the historical standard deviations will be calculated

Settings: reportFilter

- `opts.trendGapVars` List of variables whose trend/gap decomposition will be plotted in the report
- `opts.shockDecompVars` List of variables whose shock decomposition will be plotted in the report
- `opts.shockDecompGroups` A structure defining the contents of the shock groups in the shock decomposition charts

Settings: calcHistForecast

- `opts.histForecast.range` Range over which the historical forecasts will be calculated
- `opts.histForecast.horizon` The largest horizon for which forecast error statistics will be calculated (in `reportHistForecast.m`)
- `opts.histForecast.variables` List of variables for which the historical forecast plots (a.k.a. "hairy charts") will be created and the forecast error statistics will be calculated
- `opts.histForecast.exogvars` A list of variables/observables/shocks defining the exogenous variables in the historical forecast calculation (see the next slide for details)

Settings: calcHistForecast.exogvars

- It must be a string array with three columns
- In each row
 - the first element is the name of the variable to be exogenized (i.e. fixed)
 - the second element is the name of the corresponding measurement variable
 - the third element is the name of the corresponding shock
- For unobserved variables, the full sample filtered values will be fixed in the Kalman filter
- If the third element is not empty, the observed or the full sample filtered values will be fixed in the forecast

Section contents

Codes

Actions

Reports

List of codes

- Type `help codes` in the MATLAB command window to see the list of the actions and reports
- You should get this list

```
calcForecast  
calcHistForecast  
filterHistory  
readData  
readModel  
reportCompareData  
reportCompareForecast  
reportFilter  
reportForecast  
reportForecastChangeDecomp  
reportHistForecast  
reportModel
```

- Type `help codes.<codeName>` to get detailed help on each of these codes

Actions: readData

- `readData.m` reads and processes the data from the Excel file `data/Rwametric.xlsx`. Results are save in `data.mat`. There are four databases in this file
 - `dbObs` is the database of the observed variables used in the Kalman filter
 - `dbObsTrans` is the database of the transformed observed variables, e.g. growth rates
 - `dbAux` is the database of the variables necessary to evaluate the equations of the auxiliary model (reporting equations)
 - `db` is the database of unprocessed data as read from the source files
- Type `help codes.readData` to get a more detailed description and the list relevant options

Actions: readModel

- `readModel.m` reads the model equations from the model file, sets the parameter values, calculates the steady state and the first order reduced from of the model
- The resulting model object `m` is saved in `model.mat`
- Type `help codes.readModel` to get a more detailed description and the list relevant options

Actions: filterHistory

- `filterHistory.m` estimates the unobserved variables and shocks of the model using the Kalman filter, optionally with expert judgments (a.k.a. historical tunes)
- Results are saved in `filter.mat` There are tree databases
 - `dbFilt` the database of filtered variables, for all parameterizations
 - `dbObs` the combined database of observed variables and historical tunes
 - `dbEqDecomp` the database of equations decompositions, for all parameterizations
 - `dbShockDecomp` the database of shock contributions, for all parameterizations
- Type `help codes.filterHistory` to get a more detailed description and the list relevant options

Actions: calcForecast

- `calcForecast.m` calculates the forecast with optional expert judgments (a.k.a. forecast tunes)
- If there are multiple parameterizations, the first one will be used
 - It is possible to modify the model parameters in the scenario definition files, but all unmodified parameters will be taken from the first parametrization
- Results are saved in `forecast.mat`
 - `dbFcast` is the forecast database, for all scenarios
 - `dbTunes` is a cell array of the tunes for each scenario
 - `dbInit` is a database of the initial conditions
 - `m` is the forecast model, for all scenarios
 - `pln` is a cell array of the simulation plan for each scenario
 - `dbEqDecomp` the database of equations decompositions, for all parameterizations
- Type `help codes.calcForecast` to get a more detailed description

Actions: calcHistForecast

- `calcHistForecast.m` calculates historical forecasts with the model
- The algorithm loops over each period t in `histForecast.horizon`
- In each iteration
 - It runs the Kalman filter over the range starting in `filterHistory.range(1)` and ending in period $t - 1$ to get *real time estimates* of the unobserved variables
 - It calculates the forecast over a range starting in period t , using the real time estimates of the initial conditions
 - The end of the forecast range is the last period in which all fixed values are available (see the description of settings for further explanation)
- The resulting `dbFcast` database is saved in `histForecast.mat`
- Type `help codes.calcHistForecast` to get a more detailed description and the list relevant options

Reports: reportModel

- `reportModel.m` loads the model, the observed and filtered data, and creates a PDF report depicting selected model properties and evaluation indicators. The possible indicators are:
 - Steady state table
 - Impulse response charts
 - Variance decomposition charts
 - Standard deviation comparison tables
- The PDF report is saved as `modelReport.pdf`
- Type `help codes.reportModel` to get a more detailed description and the list relevant options

Reports: reportFilter

- `reportFilter.m` loads the model, the observed and filtered data, and creates a PDF report depicting selected results of the Kalman filter.
- The charts can show the following:
 - charts of the observed data used by the filter
 - charts of estimated trends and gaps
 - charts and tables of decompositions of equations
 - charts of shock decompositions
 - charts and table of estimated shocks
- The PDF report is saved as `filterReport.pdf`
- Type `help codes.reportFilter` to get a more detailed description and the list relevant options

Reports: reportForecast

- `reportForecast.m` loads the forecast results, and creates a PDF report with tables and charts of selected variables, together with the extended versions of the trend/gap charts, equations decompositions, and shock charts which are shown in the history report. The charts/tables show the forecasts for all scenarios.
- The PDF report is saved as `forecastReport.pdf`
- Type `help codes.reportForecast` to get a more detailed description and the list relevant options

Reports: reportCompareData

- `codes.reportCompareData` loads the current and comparison observed data, and creates a PDF report showing the series side-by-side in charts. The report contains comparison charts for the both series in dbObs, as well as in dbAux.
- The PDF report is saved as `dataCompareReport.pdf`
- Type `help codes.reportCompareData` to get a more detailed description and the list relevant options

Reports: reportCompareForecast

- `codes.reportCompareForecast` loads the current and comparison forecasts, and creates a PDF report showing the series side-by-side in charts. The report contains comparison charts for the both series in dbObs, as well as in dbAux. Trend/gap charts and shock charts/tables are also added.
- The PDF report is saved as `forecastCompareReport.pdf`
- Type `help codes.reportCompareForecast` to get a more detailed description and the list relevant options

Reports: reportForecastChangeDecomp

- `codes.reportForecastChangeDecomp` loads the forecast results of the current and comparison rounds, and decomposes the difference of the forecasts into the effect of a) *model change*, b) *initial condition change*, and c) *tune (i.e. shock) change*. The latter is further decomposed in to the contribution of individual shocks. The decompositions are shown in a PDF report.
- The PDF report is saved as `forecastChangeDecompReport.pdf`
- Type `help codes.reportForecastChangeDecomp` to get a more detailed description and the list relevant options

Reports: reportHistForecast

- `reportHistForecast.m` creates a PDF report showing the historical forecasts in charts ("hairy charts"), and the main forecast error statistics in table (for all model parametrizations).
- The PDF report is saved as `histForecastReport.pdf`
- Type `help codes.reportHistForecast` to get a more detailed description

Section contents

Utilities and reporting functions

List of utilities

- Type `help codes.utils` in the MATLAB command window to see the list of utility functions
- You should get this list

```
calcSubplotSize  
checkFile  
closeFigures  
loadResult  
processOptions  
saveCsv  
saveEmf  
saveReport  
saveResult  
selectFromList  
writeMessage  
writeSetparam
```

- Type `help codes.utils.<utilityName>` to get detailed help on each of these codes

List of reporting functions

- Type `help codes.reporting` in the MATLAB command window to see the list of reporting functions
- You should get this list

```
addChartPage  
addDecompCharts  
addDecompTables  
addParameterComparison  
addShockDecompositions  
addTablePage  
addTrendsAndGaps  
addVariableCharts  
barContrib  
calcDecompositions  
createParamLegend  
createScenarioLegend  
tableContrib
```

- Type `help codes.reporting.<utilityName>` to get detailed help on each of these codes